

Class Notes for Introduction to Discrete Mathematics

Mark Siggers

ver. 2026/05/22

These notes are for a lower undergraduate first class on discrete mathematics following the text “Introduction to Discrete Mathematics’ by Matoušek and Nešetřil.

The notes frequently refer to the text, as ‘the text’, making it necessary to have the text to follow the notes.

1 Introduction and Basic Concepts

1.1 Problems

The course is focused on problem solving and deals mainly with problems of combinatorial counting, and of graphs. See the text for some nice examples of such problems.

1.2 Numbers and Sets: Notation

We will use the following notation frequently. If you are unfamiliar with any of them, see the text for further explanation.

$\mathbb{N} = \{1, 2, 3, \dots\}$ is the set of *natural numbers*.

$\mathbb{Z} = \{0, \pm 1, \pm 2, \dots\}$ is the set of *integers*.

$\sum_{i=1}^3 i = 1 + 2 + 3$ is used for sums.

$\prod_{i=1}^{10} i = 1 \cdot 2 \cdot \dots \cdot 10$ is for products.

The *floor* $\lfloor x \rfloor$ is the greatest integer less than x .

The *ceiling* $\lceil x \rceil$ is the least integer greater than x .

$X \cup Y$ is the *union* of the sets X and Y .

$X \cap Y$ is their *intersection*.

$$X \setminus Y = \{x \in X \mid x \notin Y\}.$$

If S is a subset of X then we write $S \subset X$.

An *ordered set*, or *tuple* is written (a, b) . It is different from the unordered set $\{a, b\}$.

The *product* of X and Y is

$$X \times Y = \{(x, y) \mid x \in X, y \in Y\}.$$

The *cardinality* $|X|$ of X is the number of elements in it. (Most of our sets are finite, so this is a much a definition as we need.)

The *power set* 2^X of a set X is the set of subsets of X .

We should know the following set rules:

- i) Commutativity: $X \cap Y = Y \cap X$ and $X \cup Y = Y \cup X$
- ii) Associativity: $X \cap (Y \cap Z) = (X \cap Y) \cap Z$ (and its dual version).
- iii) Distributivity: $X \cap (Y \cup Z) = (X \cap Y) \cup (X \cap Z)$ (and dual).
- iv) DeMorgan: $X \setminus (A \cup B) = (X \setminus A) \cap (X \setminus B)$

Problems from the Text

Section 1.2: 1, 2, 4, 5, 6, 7 (The underlined problems are the ones that are more likely on the quiz– they are easily done in 2 minutes.)

1.3 Mathematical Induction

A proof by induction is used to prove a statement parametrized by a countable set.

Problem 1.3.1. Prove that

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

is true for all natural numbers n .

Problems from the Text

Section 1.3: 1, 2, 3a, 4, 5, 6, 8, 9, 11, 12, 14

Question 4 uses that $\sin(\alpha + \beta) = \sin \alpha \cos \beta + \sin \beta \cos \alpha$ and that $\cos(\alpha + \beta) = \cos \alpha \cos \beta - \sin \alpha \sin \beta$.

1.4 Functions

A *function* or a *map* $f : A \rightarrow B$ from a set A to a set B assigns an element $f(a) \in B$ to every element a of A .

It is often defined algebraically with a formula:

$$f(x) = x^2 + 1.$$

This same function can be described with the 'maps to' notation:

$$f : x \mapsto x^2 + 1.$$

One can also describe a function simply by listing the pairs $(a, f(a))$ or by drawing all these pairs in $\mathbb{R}^2$.

For a set $X \subset A$ we write

$$f(X) = \{f(x) \mid x \in X\}.$$

The *composition* of functions $f : A \rightarrow B$ and $g : B \rightarrow C$ is the function

$$g \circ f : A \rightarrow C : a \mapsto g(f(a)).$$

A function $f : A \rightarrow B$ can be

- *one-to-one* or *injective*: $f(x) = f(y) \Rightarrow x = y$;
- *onto* or *surjective*: $\forall b \in B \exists a \in A \text{ s.t. } f(a) = b$; or
- *bijective*: injective and surjective.

The bijective function will be important to us as we will use it in the method of 'counting by bijection'. This uses the fact that if there is a bijection between two sets, then they have the same cardinality.

Example 1.4.4 of the text asks us to count the number of 8 digit sequences:

$$(x_1, x_2, \dots, x_8)$$

made up of the digits from 0 to 9. There is a pretty obvious bijection that maps these sequences to the integers from $\{1, \dots, 10^8\}$ and so it is easy to count that there are 10^8 such sequences. They then go on to count how many of these sequences have an even number of odd digits. How many do?

Often, to show that a function is bijective, it is useful to recall that a function $f : A \rightarrow B$ is bijective if and only if there is a function $f^{-1} : B \rightarrow A$ such that $f \circ f^{-1}(b) = b$ and $f^{-1} \circ f(a) = a$ for all $a \in A$ and $B \in B$. Such f^{-1} is called the *inverse* of f . You will prove this in the following problems.

Problems from the Text

Section 1.4: 1, 2, 3, 4, 5, 6

1.5 Relations

A *relation* is a set of ordered pairs:

$$R \subseteq X \times Y.$$

If $(x, y) \in R$ we write xRy .

If $R \subseteq X \times X$ for some set X , which is usually the situation, then R is a *relation on X* .

One can represent a relation by a $\{0, 1\}$ matrix $M = [m_{ij}]$ whose columns are indexed by elements of X and whose rows are indexed by elements of Y . For a pair (x, y) the entry m_{xy} is a 1 if $(x, y) \in R$ and is 0 otherwise.

Alternately, one can represent a relation by a digraph: every element in $X \cup Y$ is a vertex, and if $(x, y) \in R$ there is an arc from x to y .

As with a function, relations can be composed: if $R \subset X \times Y$ and $S \subset Y \times Z$ then $R \circ S$ is the relation on $X \times Z$ defined

$$R \circ S = \{(x, z) \mid \exists y \in Y [(x, y) \in R \text{ and } (y, z) \in S]\}.$$

Recalling function notation, and that a function can be viewed as a relation, you might think this should be written as $S \circ R$. Be careful, it isn't.

If R is a relation on X and Y is a subset of X , then the relation

$$R|_Y = \{(a, b) \in R \mid a, b \in Y\} = R \cap (Y \times Y)$$

is called the *restriction of R to Y* .

Problem 1.5.1. Can you find some explanation for the difference in notation between compositions of functions and of relations?

Problems from the Text

Section 1.5: 1, 2, 3, 4, 5

1.6 Types of Relations

A relation on a set X is

- *reflexive* if xRx for all $x \in X$,
- *symmetric* if $xRy \Rightarrow yRx$,
- *antisymmetric* if $(xRy \wedge yRx) \Rightarrow x = y$,

- *transitive* if $(xRy \wedge yRz) \Rightarrow xRz$

For a relation R , the *opposite* relation R^{-1} is the relation

$$R^{-1} = \{(y, x) \mid (x, y) \in R\}.$$

The *diagonal relation* Δ_X on X is

$$\Delta_X = \{(x, x) \mid x \in X\}.$$

Theorem 1.6.1. *A relation R on X is*

- i) reflexive iff $\Delta_X \subset R$,*
- ii) symmetric iff $R = R^{-1}$,*
- iii) antisymmetric iff _____,*
- iv) transitive iff _____,*

A relation that is reflexive symmetric and transitive is an *equivalence relation*.

An ordering R such that for every $x, y \in X$ either xRy or yRx is a *linear* (or *total*) *ordering*.

For an equivalence relation R on X and a element x , the *equivalence class* of x is

$$R[x] = \{y \in X \mid yRx\}.$$

Proposition 1.6.2. *Let R be an equivalence relation on X , then*

- i) $\forall x \in X, x \in R[x]$,*
- ii) $\forall x, y \in X$, either $R[x] = R[y]$ or $R[x] \cap R[y] = \emptyset$,*
- iii) the classes $R[x]$ uniquely define R .*

A *partition* of a set X is a family $\mathcal{P}$ of subsets of X such that for all $x \in X$ there is a unique subset $S \in \mathcal{P}$ such that $x \in S$.

The above proposition shows that equivalence relation on X defines a partition, and every partition of X defines a relation.

A relation that is reflexive antisymmetric and transitive is an *ordering* (or a *partial ordering*). We look at these in more detail in the next section.

Problem 1.6.3. Let R be an equivalence relation on X and $Y \subset X$. Show that the restriction $R|_Y$ is an equivalence relation. Show that the restriction of an ordering to a subset is an ordering.

2 Orderings

Recall that a (partial) ordering of a set X is a reflexive, antisymmetric, transitive relation on X . In this chapter we look at orderings in more detail.

2.1 Depictions of orderings

Orderings we are familiar with are the usual ordering $\leq$ of the $\mathbb{N}$ or $\mathbb{Z}$ or $\mathbb{R}$.

Problem 2.1.1. Show that the relation R on $\mathbb{N}$ defined by aRb if $a \leq b$ is an ordering.

Because of the anti-symmetry, it is usual to use anti-symmetric notation $\leq$ or $\preceq$ for an ordering. When we do so the opposite relations $\leq^{-1}$ and $\preceq^{-1}$ are denoted $\geq$ and $\succeq$ respectively, and the ‘strict’ versions $\leq \setminus \Delta_{\leq}$ and $\preceq \setminus \Delta_{\preceq}$ of the relations are denoted $<$ and $\prec$ respectively.

Problem 2.1.2. Is the strict version $<$ of an ordering $\leq$ an ordering?

Problem 2.1.3. Find an ordering $\preceq$ of $\mathbb{N}^2$.

Two elements x and y are *comparable* under an ordering $\leq$ if either $x \leq y$ or $y \leq x$; otherwise they are *incomparable*.

A *linear (or total) ordering* on X is an ordering $\leq$ on X that every pair of elements is comparable.

Problem 2.1.4. Was the ordering you gave of $\mathbb{N}^2$ above a linear ordering?

We often use the optional ‘partial’ in front of ‘ordering’ when we want to emphasise that it is not necessarily a linear ordering. A *poset* $(X, \leq)$ is a set X with a **partial ordering** $\leq$ on X .

There are some other non-linear orderings that we are quite familiar with.

Problem 2.1.5. Show that the set of positive integers $\mathbb{Z}^+$ ordered by divisibility

$$a \preceq b \text{ if } a|b,$$

is a poset.

Show that the set of subsets of a set S ordered by inclusion:

$$A \prec B \text{ if } a \subseteq b,$$

is a poset.

As a poset is a relation, we have seen how to draw it, but the picture has a lot of redundant information. Removing this, we get another picture of a poset that is a lot clearer.

Definition 2.1.6. Let $\leq$ be a partial ordering on X . For elements $x, y \in X$, x is an *immediate predecessor* of y , or is *covered* by y ,

- $x \leq y$, and
- $(x \leq t \leq y) \Rightarrow (x = t \text{ or } x = y)$.

In this case, we write $x \triangleleft y$.

In the *Hasse diagram* or *cover diagram* of a poset $(X, \leq)$ we draw the elements of X , and we put an arrow from x to y if $x \triangleleft y$. To simplify things further, we usually draw an edge instead of an arrow and denote the direction by of the arrow $x \rightarrow y$ by drawing x higher on the paper than y .

This diagram is enough to represent the poset because $x \leq y$ if and only if there is a path of edges in the diagram going down from x to y . We prove this now.

Proposition 2.1.7. For a finite poset $(X, \preceq)$, and $x, y \in X$, we have that $x \preceq y$ if and only if there is $n \in \mathbb{N}$ and elements $x_1, x_2, \dots, x_n \in X$ such that

$$x = x_1 \triangleleft x_2 \triangleleft \dots \triangleleft x_n = y.$$

Proof. See page 46 of the text for the proof.

□

Problems from the Text

Section 2.1: 1,2,3,4

2.2 Orderings and linear orderings

Definition 2.2.1. A linear ordering $\preceq$ on X is a (*linear*) *extension* of an ordering $\leq$ on X if for all $x, y \in X$

$$x \leq y \Rightarrow x \preceq y.$$

Problem 2.2.2. Show that the usual ordering of $\mathbb{N}$ extends the divisibility ordering on $\mathbb{N}$. Are there any other extensions of the divisibility ordering on $\mathbb{N}$?

Problem 2.2.3. Find a linear extension of the ordering $\preceq$ on $\mathbb{N}^2$ that is defined by

$$(a, b) \leq (a', b') \text{ if } a \leq b \text{ and } a' \leq b'.$$

In this section we will show that every finite poset has a linear extension. We need some definitions.

Definition 2.2.4. Let $(X, \leq)$ be a poset. An element $x \in X$ is

- *minimum* if $\forall y \in X, x \leq y$,
- *minimal* if $\nexists y \in X$ with $y \leq x$,
- *maximum* if $\forall y \in X, y \leq x$,
- *maximal* if $\nexists y \in X$ with $x \leq y$.

Note that a minimum element is minimal, but minimal elements are not generally minimum.

Theorem 2.2.5. *Every finite poset has at least one minimal element.*

Proof. See page 49 of the text.

□

Theorem 2.2.6. *Every finite partial ordering $(X, \leq)$ has a linear extension.*

Proof. The proof is by induction on $n = |X|$. Clearly the Theorem holds for $n = 1$, as there is only one ordering on a single element, and it is linear.

Now assume that the theorem holds for posets of size up to $n - 1$ and let $|X| = n$. By the previous theorem, there is a minimal element x_0 of X . Let $X' = X \setminus x_0$ and let $\leq'$ be the restriction of $\leq$ to X' . By Problem 1.6.3 $\leq$ is an ordering, and by the induction assumption it has an extension $\preceq'$ to a linear ordering of X' . We define the ordering $\preceq$ on X by

$$x \preceq y \text{ if } x \preceq' y \text{ or } x = x_0.$$

Finish the proof by showing this is a linear extension of $\leq$. (See page 50 of the text for the details.) □

Problem 2.2.7. In the above proof we said that there is only one ordering on a one element set. How many orderings are there on a 2 element set, and how many of them are linear? How many linear orderings are there on an n element set? How many orderings? (This last one might be difficult. We will see the answer in Chapter 8.)

Problems from the Text

Section 2.2: 1,2,3,4,6,7,8,10

2.3 Orderings by Inclusion

In this section we show that every poset can be viewed as the inclusion poset of some family of sets.

Definition 2.3.1. Let $P = (X, \leq)$ and $P' = (X', \leq')$ be posets. A mapping $f : X \rightarrow X'$ is an embedding of P into P' if it is injective and for all $x, y \in X$

$$f(x) \leq' f(y) \iff x \leq y.$$

Problem 2.3.2. Show that the usual linear ordering on the set $\{0, 1, 2, \dots, n\}$ embeds into $(2^{[n]}, \subseteq)$ (the set $2^{[n]}$ with the inclusion ordering). Does it embed into $(2^{[n-1]}, \subseteq)$?

Theorem 2.3.3. For any ordered set $P = (X, \leq)$, the map

$$f : X \rightarrow 2^X : x \mapsto L_x = \{y \in X \mid y \leq x\}$$

is an embedding of P into $(2^X, \subseteq)$.

Proof. See page 53 of text.

Note

Recall to show that $A \subset B$ we can show that for any $a \in A$, $a \in B$.

□

Consider the set $B_n = \{0, 1\}^n$ of n element binary strings: n elements strings of the digits 0 and 1. For two such strings $x = (x_1, x_2, \dots, x_n)$ and $y = (y_1, y_2, \dots, y_n)$ let

$$x \preceq y \iff \forall i \in [n], x_i \leq y_i.$$

This is an ordering, and $\mathcal{B}_n = (B_n, \preceq)$ is called the boolean lattice.

Problem 2.3.4. Show that the boolean lattice $\mathcal{B}_n$ and the poset $(2^{[n]}, \leq)$ are isomorphic.

Problems from the Text

Section 2.3: 1,2,3,4,5,7,8

Hint: For question 4, try an induction. Remove a minimal element, get the embedding, then map the minimal element to a new prime that isn't used yet. You will have to then re-embed the elements above the new element.

2.4 Large implies tall or wide

Let $P = (X, \leq)$ be a poset.

Definition 2.4.1. A subset $A \subset X$ is a *chain* if every pair of elements in A is comparable, and is an *antichain* if no pair of elements in A is comparable.

Problem 2.4.2. Show that the set of all minimal elements of a poset is independent.

Definition 2.4.3. The *height* $\omega(P)$ of a poset P is the size of the largest chain in P . The *width* $\alpha(P)$ is the size of the largest antichain.

Problem 2.4.4. Find $\omega(\mathcal{B}_4)$ and $\alpha(\mathcal{B}_4)$.

Theorem 2.4.5. For every finite poset $P = (X, \leq)$,

$$\alpha(P) \cdot \omega(P) \geq |X|.$$

Proof. See page 56 of the text. □

For a sequence $(x_1, x_2, \dots, x_N)$ of integers, a subsequence $(x_{i_1}, x_{i_2}, \dots, x_{i_n})$ is monotone if either

$$x_{i_1} \leq x_{i_2} \leq \dots \leq x_{i_n}$$

or

$$x_{i_1} \geq x_{i_2} \geq \dots \geq x_{i_n}.$$

Theorem 2.4.6 (Erdős, Szekeres). Any sequence $X = (x_1, \dots, x_N)$ of $N = n^2 + 1$ real numbers has a monotone subsequence of length at least $n + 1$.

Proof. It is easy to check that the relation $\preceq$ on the set $[N] = \{1, \dots, N\}$ defined as follows is an ordering: set $i \preceq j$ if $i \leq j$ and $x_i \leq x_j$.

By the previous theorem we have that

$$\alpha([N], \preceq) \cdot \omega([N], \preceq) \geq N = n^2 + 1$$

so either $\alpha([N], \preceq) \geq n + 1$ or $\omega([N], \preceq) \geq n + 1$.

But any chain in $([N], \preceq)$ gives an increasing subsequence of X and any antichain $([N], \preceq)$ gives a decreasing subsequence of X . □

Problem 2.4.7. Fill in the details of this proof. Show that $\preceq$ is an ordering of $[N]$. Show that a chain or antichain in $([N], \preceq)$ corresponds to an increasing or decreasing sequence in X , respectively.

Problems from the Text

Section 2.4: 1, 2, 3, 4, 5, 6

For # 3, it should be 16, not 17. Why?

3 Combinatorial Counting

3.1 Functions and subsets

Consider the following Jellybean problem.

Problem 3.1.1. In a big bag of jellybeans, there are 25 different colours of jellybean (and lots of each colour). How many different ways can you give one jellybean to each of 5 kids?

Clearly, there are three ways: you can give the kids the jellybeans in small wrapped boxes, or by putting them under their pillows when they are asleep, or tossing them across the room into their gaping mouths. But also, clearly, this isn't what the question is asking.

A 'way' to give one jellybean to each kid is a function f from the set of K of kids, to the set C of colours. The above problem is thus the same as:

Problem 3.1.2. Let $|K| = 5$ and $|C| = 25$. How many different functions are there from K to C .

We've mentioned the following before, but we prove it now by restating it as a problem of counting functions.

Proposition 3.1.3. *For a finite set X , there are $2^{|X|}$ different subsets.*

Proof. Each subset $S \subset X$ defines a function

$$f_S : X \rightarrow \{0, 1\} : x \mapsto \begin{cases} 0 & \text{if } x \notin S \\ 1 & \text{if } x \in S, \end{cases}$$

called the *characteristic function* of S .

On the other hand, each function $f : X \rightarrow \{0, 1\}$ defines a subset

$$S_f = \{x \in X \mid f(x) = 1\}$$

of X . We can check that that $f_{S_f} = f$ for each function $f : X \rightarrow \{0, 1\}$ and that $S_{f_S} = S$ for each $S \subset X$. So there is a bijection between the subsets of X and the functions $X \rightarrow \{0, 1\}$. As there are $2^{|X|}$ of these, we are done. $\square$

Problem 3.1.4. Check that $f_{S_f} = f$ and $S_{f_S} = S$ in the above proof.

Using the above proposition, we can prove the following with another proof by bijection.

Proposition 3.1.5. *There are $2^{|X|-1}$ subsets of X of odd size.*

Proof. There are two bijections that suggest themselves.

We could prove this by finding a bijection between the subset of X of odd size and the subsets of X of even size. Or we could find a bijection between the subset of X of odd size, and the subset of a set X of size $|X| - 1$.

Can you think of how to define one of these bijections? On page 62 of the text they do the second method. $\square$

How about some more jellybeans?

Problem 3.1.6. State the following problem as a problem about counting functions, and solve it:

How many ways can we give a jellybean to each kid if there is only one jellybean of each colour?

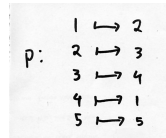
Problems from the Text

Section 3.1: 1-4,6

3.2 Permutations

A *permutation* of a set X is a bijection $X \rightarrow X$.

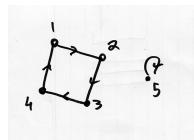
The function $p : [5] \rightarrow [5]$ shown is a permutation.



It can be represented by

$$p = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 2 & 3 & 4 & 1 & 5 \end{pmatrix} \text{ or simply } p = (23415).$$

Or we can draw it as a relation



which suggests the following cycle notation:

$$p = ((1234)(5)).$$

Problem 3.2.1. How many permutations are there of an n element set?

Problems from the Text

Section 3.2: 1-3, 6ab, 8

3.3 Binomial Coefficients

Recall the binomial coefficient

$$\binom{n}{k} = \frac{n(n-1)\dots(n-k)}{k(k-1)\dots(2)(1)} = \frac{n!}{(n-k)!k!}$$

counts the number of k -element subsets of an n element set.

It makes sense to define, for a set X

$$\binom{X}{k} = \text{the set of } k\text{-element subsets of } X.$$

So $|\binom{X}{k}| = \binom{|X|}{k}$.

The binomial coefficient is useful in counting jellybeans.

Problem 3.3.1. There (an unlimited number of each of) five colours of jellybean. How many ways can you choose 12 jellybeans?

Problem 3.3.2. How many integer solutions are there to the equation

$$x_1 + x_2 + x_3 + x_4 = 15,$$

with $x_i \geq 0$ for each i . How many with $x_i \geq 1$ for each i ?

Properties of the Binomial Coefficient.

- $\binom{n}{k} = \binom{n}{n-k}$
- $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$.
- $\binom{n}{k}\binom{k}{m} = \binom{n}{m}\binom{n-m}{k-m}$
- The Binomial Theorem: $(1+x)^n = \sum_{i=0}^n \binom{n}{i}x^i$

Problem 3.3.3. Use the binomial theorem to show that there are 2^n different subsets of the set $[n]$.

The following identity is a little less common than those above, but it has a pretty proof.

Theorem 3.3.4.

$$\binom{2n}{n} = \sum_{i=0}^n \binom{n}{i}^2$$

Proof. Page 72 of text.

□

3.3.1 The Multinomial Coefficient

Problem 3.3.5. You have two blue jellybeans, four green, one red, two yellow, and three white. How many ways can you order them?

If you said $12!$, you are misunderstanding deliberately. Go to your room!

Where $n = \sum_{i=1}^d k_i$, the multinomial

$$\binom{n}{k_1, k_2, \dots, k_d} = \frac{n!}{k_1! \cdots k_d!},$$

counts the number of ways we can order a multiset of n elements where there are k_i indistinguishable elements of type i .

Problems from the Text

Section 3.3: 2,3,4,5,6,7,8,9,11,15,17,18

3.4 Estimates

If we wanted to compute the n^{th} harmonic number

$$H_n = \sum_{i=1}^n \frac{1}{i}$$

exactly, we would have to compute the whole sum. There is no good closed formula for it. But if we only want to estimate it, we can do pretty well with closed formulae.

Here is a pretty easy upper bound. We get it by collecting terms cleverly:

$$H_n = \underbrace{\frac{1}{1}}_{G_1} + \underbrace{\frac{1}{2} + \frac{1}{3}}_{G_2} + \underbrace{\frac{1}{4} + \frac{1}{5} + \dots + \frac{1}{7}}_{G_3} + G_4 + \dots + G_{\lfloor \log_2 n \rfloor} + \overbrace{G_{\lceil \log_2 n \rceil}}^{\text{some of}}.$$

As

$$G_i = \sum_{j=2^{i-1}}^{2^i-1} 1/j < 2^{i-1} \cdot 1/2^{i-1} = 1$$

we have that $H_n < \lceil \log_2 n \rceil$.

On the other hand,

Problem 3.4.1. Show that $G_n > 1/2$.

so we get that $H_n > \frac{1}{2} \lfloor \log_2 n \rfloor$. So $\log_2 n$ is a pretty good estimate for H_n . We write $H_n = O(\log_2 n)$.

Definition 3.4.2. For real functions f and g we write $f = O(g)$ and say that f is *big-oh* of g if there exists some N and $c > 0$ such that

$$x > N \rightarrow |f(x)| < c|g(x)|.$$

That f is big-O of g means that asymptotically (as x gets big), f is not much bigger than g . Alternately, that asymptotically, the graph of f is not above the graph of g , if we zoom far enough out.

It is not too hard to show that if

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = c$$

for some c then $f = O(g)$.

Definition 3.4.3. If $\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = 0$ then we write $f = o(g)$ and say f is *little oh* of g .

Definition 3.4.4. For real functions f and g we write $f \preceq g$ if there exists some N such that $n > N$ implies $f(n) \leq g(n)$.

Note that $f \preceq g$ is a little stronger than $f = O(g)$.

Observe that for any positive reals c_1 , c_2 , and c_3 we have

$$\log_{c_1} n \preceq n^{c_2} \preceq c_3^n.$$

Problems from the Text

Section 3.4: 1-3(abd),4,6

3.5 Estimating $n!$

How big is $n!$? It is $O(n^c)$ for some c ? Is it $O(c^n)$ for some c ? Nope, even bigger. It is $O(n^n)$.

It is simple to show that

$$2^{n-1} \leq n! \leq n^n.$$

It is not too hard to show that

$$\left(\frac{\sqrt{n}}{\sqrt{2}}\right)^n \leq n! \leq \left(\frac{n}{\sqrt{2}}\right)^n.$$

Gauss showed

$$\sqrt{n}^n \leq n! \leq \left(\frac{n+1}{2}\right)^n.$$

In fact the bounds can be improved to the following.

$$e \left(\frac{n}{e}\right)^n \leq n! \leq n \cdot e \left(\frac{n}{e}\right)^n. \quad (1)$$

If we want an even better estimation, we can use Stirlings approximation. This is quite hard to prove.

$$n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n.$$

Problems from the Text

Section 3.5: 1,2,9a,11

3.7 Inclusion and Exclusion

Consider the following problem.

Problem 3.7.1. Out of a group of students,

- 30 do math, 15 do statistics, 20 do physics
- 10 do math and physics, 4 do math and stats, 8 do stats and phys
- 3 do all three subjects.

How many students do at least one of the three subjects?

More generally, one would use the principle of inclusion and exclusion.

Theorem 3.7.2. Where $A_1, \dots, A_d$ are subsets of some set X :

$$|\bigcup_{i=1}^d A_i| = \sum_{k=1}^d (-1)^{k-1} \sum_{I \in \binom{[d]}{k}} |\bigcap_{i \in I} A_i|.$$

Problems from the Text

Section 3.7: 1,3,4,5,6

3.8 Derangements

Sometimes to count a set, it is easier to count the complement.

Problem 3.8.1. How many numbers in the interval $[100]$ are relatively prime to 3? (That is for how many numbers n does $\gcd(3, n) = 1$.)

Problem 3.8.2. How many numbers in the interval $[100]$ are relatively prime to 15?

The following is a dual form of the principle of Inclusion and Exclusion.

Theorem 3.8.3. Where $A_1, \dots, A_d$ are subsets of some set X , and $|\cap_{i \in \emptyset} A_i| = |X|$, the number of elements in **none** of the sets A_i is

$$|\bigcap_{i=1}^d \overline{A_i}| = \sum_{k=0}^d (-1)^k \sum_{I \in \binom{[d]}{k}} |\bigcap_{i \in I} A_i|.$$

Proof. Indeed, by DeMorgan we have that $\bigcap_{i=1}^d \overline{A_i} = \overline{\bigcup_{i=1}^d A_i}$ so has size $|X| - |\bigcup_{i=1}^d A_i|$.

Thus

$$\begin{aligned} |\bigcap_{i=1}^d \overline{A_i}| &= |X| - |\bigcup_{i=1}^d A_i| = |X| - \sum_{k=1}^d (-1)^{k-1} \sum_{I \in \binom{[d]}{k}} |\bigcap_{i \in I} A_i| \\ &= |X| + \sum_{k=1}^d (-1)^{\textcolor{red}{k}} \sum_{I \in \binom{[d]}{k}} |\bigcap_{i \in I} A_i| \\ &= \sum_{k=\textcolor{red}{0}}^d (-1)^k \sum_{I \in \binom{[d]}{k}} |\bigcap_{i \in I} A_i| \end{aligned}$$

□

Definition 3.8.4. A *derangement* of $[n]$ is a permutation σ of $[n]$ such that for all $i \in [n]$, $\sigma(i) \neq i$.

Problem 3.8.5. How many derangements are there of $[n]$?

Problems from the Text

Section 3.8: 1,2,3,6,7, 8ab

4 Graphs: An introduction

4.1 The notion of a graph; isomorphism

Definition 4.1.1. A *graph* G is a pair (V, E) where V is a set and E is a set of 2 element subsets of V . The elements of V are called *vertices* and the elements of E are called *edges*. We often write $V(G)$ and $E(G)$ for the sets of vertices and edges of a given graph G .

A graph is an irreflexive symmetric relation, and is represented by a picture.

If $\{u, v\}$ is an edge of a graph G we say that u and v are *adjacent* or *neighbours*, and we write $u \sim v$.

Some common named graphs are

- The *clique* or *complete graph* K_n on n vertices:

$$V(K_n) = [n], \quad E(K_n) = \{\{i, j\} \mid i, j \in [n], i \neq j\}$$

- The *n-cycle* C_n :

$$V(C_n) = [n], \quad E(C_n) = \{\{i, i+1 \bmod n\} \mid i \in [n]\}$$

- The *n-path* P_n :

$$V(P_n) = [n], \quad E(P_n) = \{\{i, i+1\} \mid i \in [n-1]\}$$

- The *complete bipartite graph* $K_{m,n}$ on the sets M and N with $|M| = m$ and $|N| = n$:

$$V(K_{m,n}) = M \cup N, \quad E(K_{m,n}) = \{\{u, v\} \mid u \in M, v \in N\}$$

A graph $G = (V, E)$ is *bipartite* if there is a partition of V into two sets A and B such that all edges are between A and B .

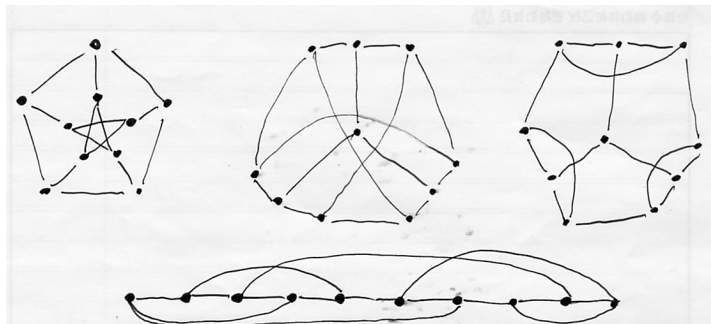
Usually, the name of a vertex has no importance. So we only consider graph upto isomorphism.

Definition 4.1.2. An *isomorphism* $f : G \rightarrow G'$ from a graph $G = (V, E)$ to a graph $G' = (V', E')$ is a bijection $f : V \rightarrow V'$ such that

$$(a \sim b) \iff (f(a) \sim f(b)).$$

If there exists an isomorphism $f : G \rightarrow G'$ then G and G' are *isomorphic*, written $G \cong G'$.

Problem 4.1.3. Which of the following graphs are isomorphic?



Problem 4.1.4. How many non-isomorphic graphs are there on 5 vertices?

Deciding if two graphs are isomorphic is a computationally hard problem. We can do it, but it takes a long time. Consider that someone gives you two graphs G and H , and asks you if they are isomorphic. The first thing I would do is check that they have the same number n of vertices and m of edges— isomorphic graphs would have to. If they do, then we have to go deeper.

One way we could check is to look at all maps $\phi : V(G) \rightarrow V(H)$, and check if any of them satisfy the isomorphism condition:

$$\forall u, v \in V(G) \quad (u \sim v \Rightarrow \phi(u) \sim \phi(v)).$$

For each $\phi : V(G) \rightarrow V(H)$ this takes m 'looks at H '. For each edge uv of G we have to check that $\phi(u)\phi(v)$ is an edge of H . We say this takes $O(n^2)$ time.

This is fine if the first ϕ we choose is an isomorphism, if we are not so lucky, we have to check others. To be sure that there is no isomorphism, we would have to check $|V(H)|^{|V(G)|}$ functions ϕ . This takes exponential time, unless we do something much more clever.

Well, we can solve this problem in time that is less than exponential in n , but we don't think that we can solve it in time polynomial in n .

Problems from the Text

Section 4.1: 1, 2, 4, 3 (doing 4 might help with 3b), 5, 6, 7

For questions 3 and 6 the following is important. Consider the graph K_2 having vertex set $\{1, 2\}$ and the one edge $\{1, 2\}$. The graph you get by switching the vertices, is the same graph: it has vertex set $\{1, 2\}$ and the one edge $\{1, 2\}$. **So it is not an isomorphic graph, it is the same graph.** There is only one graph on the vertex set $\{1, 2\}$ that is isomorphic to K_2 .

4.2 Subgraphs, Components, Adjacency Matrix

A graph G' is a *subgraph* of a graph G if

$$V(G') \subset V(G) \quad \text{and} \quad E(G') \subset E(G).$$

It is an *induced subgraph* if, in addition,

$$E(G) = \{\{u, v\} \in E(G) \mid u, v \in V(G'),\}$$

For a subset $A \subset V(G)$, the *subgraph induced by A* , denoted $G|_A$, is the induced subgraph of G with vertex set A .

Problem 4.2.1. How many subgraphs are there of K_5 ? How many non-isomorphic ones? How many are induced?

Problem 4.2.2. How quickly can you decide if a graph G on n vertices contains a subgraph isomorphic to K_3 ? To H ?

A *path* in G is a subgraph of G that is isomorphic to P_t for some t . It is usually described as a sequence of distinct vertices

$$v_1, v_2, \dots, v_n$$

such that $v_i \sim v_{i+1}$ for all $i \in [n-1]$. This path is a path *from v_1 to v_n* , or *between v_1 and v_n* .

Problem 4.2.3. How many paths are there in K_5 ? How many are induced?

A *walk* between v_1 and v_n in G is a sequence of **not necessarily distinct** vertices

$$v_1, v_2, \dots, v_n$$

such that $v_i \sim v_{i+1}$ for all $i \in [n-1]$.

Problem 4.2.4. How many walks are there in K_5 ?

A graph G is *connected* if for every two vertices $u, v \in V(G)$ there is a walk in G between u and v .

Problem 4.2.5. Show that if there is a walk in G between u and v , then there is a path between u and v .

One can define an equivalence relation $\sim_c$ on the vertex set $V(G)$ of a graph by

$$u \sim_c v \text{ if there is a walk from } u \text{ to } v.$$

Problem 4.2.6. Show that this is an equivalence relation. (We must allow paths of length 0.)

The equivalence classes $[v]_{\sim_c}$ are called *components of G* . For a component C of G sometimes we also call $G|_C$ a component of G .

The *distance* $d_G(u, v)$ between vertices u and v is the length of the shortest path from u to v .

4.2.1 Breadth First Search

How do we decide if a graph is connected, or find the distance between two vertices? These seem like easy problems, but if I give you a graph with 1000 vertices and 5000 edges, it might not be so easy.

But we do consider these to be easy problems. Both of these questions can be answered with what is called a *breadth first search* (BFS) of a graph G .

Algorithm 4.2.7 (Breadth First Search).

Input: A graph G and a vertex $v \in V(G)$.

Output: A partition of the component of G containing v into layers L_i whose members have distance i from v .

Steps:

- i) Set $L_0 = \{v\}$, $U = V(G) \setminus L_0$, and $i = 0$.
- ii) Until U is empty, or does not decrease, do:
 - (a) Increment i . (Set $i := i + 1$.)
 - (b) Find the set L_i of vertices in U that are adjacent to vertices of L_{i-1} .
 - (c) Set $U := U \setminus L_i$.

As G is finite, this algorithm clearly finishes. If we finish with U empty, then G is connected; otherwise it is not. Any vertex in L_i has distance i from v .

Running time of BFS

Step i) of this algorithm takes constant time. Computing the time that Step ii) requires is a little trickier. It depends on how we represent the graph, and on how we 'find the vertices of U that are adjacent to vertices of L_{i-1} '.

Our definition of a graph gives us a set V of vertices and a set E of edges. With just this, an easy upper bound of the running time of the algorithm is as follows. For each $u \in U$ and each $v \in L_{i-1}$, we move u from U to L_i if $u \sim v$. We know that $|U| = O(n)$ and $|L_{i-1}| = O(n)$, so this takes $O(n^2)$. We run Step ii) $O(n)$ times, so the algorithm is $O(n^3)$.

This really seems to be over counting a lot, lets see if we can do a finer argument.

A nice way to give the information of the graph to a computer might be to give it a list of vertices V , and for each vertex v a list $N(v)$ of its neighbours. Let us assume this is how our graph G is represented.

Over all runs of Step ii) a vertex v is in only one of the L_{i-1} and we need only check the vertices in $N(v)$, and add them to L_i if they are in U . The sum of the $|N(v)|$ is $2m = 2|E(G)|$, so over all runs of Step ii) we make $O(m)$ checks

of whether a vertex is in U . If we are looking through a list U , this might take some time, but we can let each vertex point to a boolean variable which tells us if it is in U or not, so this check takes constant time. All together, all runs of Step ii) take time $O(m)$, and so the algorithm takes time $O(m)$.

As a graph can have upto $\binom{n}{2} = O(n^2)$ edges, we often just write $O(m) = O(n^2)$, but $O(m)$ is more precise. It is smaller for *sparse graphs*– graphs with relatively few edges– sparse graphs are quite common in many applications.

The way we represent a graph to a computer can really affect the running time of an algorithm, but generally only affects it by a factor of about $O(n^2)$ as most representations take space $O(n^2)$.

The next section discusses another useful representation of a graph. This will also help us count cycles in a graph.

A *cycle* in G is a subgraph isomorphic to C_t for some t .

Problem 4.2.8. How many cycles are there in K_5 ? How many three cycles.

The Adjacency Matrix

To input a graph into a computer, we can give it a list V of vertices, and a list E of edges. One might also give a list V of vertices, and for each vertex, a list $N(v)$ of its neighbours. When we get down to programming something, we have to make decisions about these things. Many computer languages are really optimised for matrix calculation– it is often useful to represent a graph as a matrix.

For a graph G with n vertices let $V(G) = \{v_1, \dots, v_n\}$. The *adjacency matrix* A_G of G is the $n \times n$ matrix $A_G = [a_{ij}]$ where

$$a_{ij} = \begin{cases} 1 & \text{if } v_i \sim v_j \\ 0 & \text{otherwise.} \end{cases}$$

Problem 4.2.9. Show that there is a walk of length 2 from v_i to v_i if and only if the $(i, i)^{th}$ entry of A_G^2 is non-zero.

Problem 4.2.10. Give a necessary and sufficient condition, using A_G for the existence of

- i) a walk of length t from v_i to v_j ,
- ii) a loop on the vertex v_i .

Problem 4.2.11. How could you use the matrix A_G^3 to count the triangles in G .

Problems from the Text

Section 4.2: 1,2,4,5,8

Problem 4.2.12. The *diameter* of a graph is the maximum over all pairs of vertices u and v of the graph, of the distances $d(u, v)$. Show that you could find the diameter of a graph in $O(n^4)$ time (use BFS). Can you do better? How could you do it with the adjacency matrix?

4.3 Graph Score

Definition 4.3.1. For a vertex v in a graph G , the *degree* $\deg_G(v)$ of v is the number of edges of G containing v . Where the vertices of G are $v_1, \dots, v_k$, the multiset

$$\{\deg_G(v_1), \deg_G(v_2), \dots, \deg_G(v_n)\}$$

is the *graph score* of G .

The graph score is not a sequence, but we make it a sequence, called the *degree sequence* of G , by writing it in non-decreasing order.

Clearly isomorphic graphs have the same degree sequence. But the converse is clearly not true.

Problem 4.3.2. Find graphs with the same degree sequence that are not isomorphic.

Here is a harder question: Is every non-decreasing sequence of positive integers a degree sequence?

A non-decreasing sequence $D = (d_1, \dots, d_n)$ is *graphic* if it is the degree sequence of some graph.

Problem 4.3.3. Show that for any graphic sequence, $\sum_{i=1}^n d_i$ is even.

There are quick numerical conditions one can check to decide if a sequence is graphic, but they are a little hard to prove. We look at an algorithmic decision process.

A graph G is a *realisation* of a sequence $D = (d_1, d_2, \dots, d_n)$ if there is a labelling of its vertices as $V = \{v_1, \dots, v_n\}$ such that for all $i \in [n]$, $\deg_G(v_i) = d_i$.

Theorem 4.3.4. (Havel-Hakimi) *If a sequence $D = (d_1, d_2, \dots, d_n)$ is graphic, then there is realisation G of D such that the v_d is adjacent to $v_{n-1}, \dots, v_{n-d_n}$.*

Before we prove this. Lets see how it allows us to decide if a sequence is graphic. If D is graphic, then it has a realisation as in the lemma. If we remove the edges from v_n then the graph we get has degree sequence

$$D' = (d_1, d_2, \dots, d_{n-d_n-1}, d_{n-d_n} - 1, \dots, d_{n-1} - 1, 0)$$

which we get by changing d_n to 0 and reducing the next d_n values by one. On the other hand, if the sequence D' is graphic, then so is D .

This gives us a recursive algorithm for deciding if a degree sequence D is graphic: keep reducing the sequence as above, reordering if necessary to keep it non-decreasing, until there are no positive degrees. This clearly terminates, as the sum of degrees is reduced at each step.

If we end with a sequence of zeros, D was graphic. If we get any negative numbers, it was not graphic.

For a graph on n vertices, we have to remove $O(n)$ vertices, and doing so requires reducing the degree of its $O(n)$ neighbours. This takes time $O(n^2)$. Alternately, we might observe that we must remove $m = |E(G)|$ edges, so it takes time $O(m)$.

Now let's prove the theorem.

Proof. Let $D = (d_1, d_2, \dots, d_n)$ be a graphic sequence. If $d_n = n - 1$ then we are done, so we may assume that $d_n < n - 1$.

For any realisation G of D , let $j(G)$ be the maximum j such that $v_j \not\sim v_n$. Of all possible realisations of D let G now be a particular one that minimises $j(G)$. (Clearly $j(G) \geq n - 1 - d_n \geq 1$, and we are done if $j(G) = n - 1 - d_n$.)

We claim that $j(G) = n - 1 - d_n$. Towards contradiction, assume that $j(G) > n - 1 - d_n$. We will show that there is another realisation G' of D with $j(G') < j(G)$.

Indeed, as $j(G) > n - 1 - d_n$, there is a neighbour v_z of v_n with $z < n - d_n \leq j$. Now v_z has an edge to v_n , v_j doesn't, and as $j > z$, $\deg(v_j) \geq \deg(v_z)$; so v_j has some neighbour v_x that is not a neighbour of v_z .

Let G' be the graph we get from G by removing the edges $\{v_n, v_z\}$ and $\{v_j, v_x\}$ and adding the edges $\{v_n, v_j\}$ and $\{v_x, v_z\}$. It has the same degree sequence, so is a realisation of D , but now $j(G') < j(G)$. $\square$

Problem 4.3.5. Decide, using Havel-Hakimi, whether or not the sequence $(5, 5, 5, 3, 2, 2, 1, 1)$ is graphic.

Problems from the Text

Section 4.3: 1,2,4,5,6,7,8, 9,10,12,13,16

4.4 Eulerian Graphs

What graphs can we draw without lifting our pencil from the paper?

For this section it will be useful to represent a walk as

$$v_0, e_1, v_1, e_2, \dots, v_{n-1}e_n, v_n$$

where $e_i = v_{i-1}, v_i$ is the edge between the vertices v_{i-1} and v_i .

Definition 4.4.1. An *Eulerian tour* of a graph is a walk that contains every vertex of the graph, and uses every edge of the graph exactly once. It is *closed* if it starts and ends at the same vertex. A graph is *Eulerian* if it has a closed Eulerian tour.

We can represent a closed tour as

$$v_0, e_1, v_1, e_2, \dots, v_{n-1}, e_n$$

where we understand that e_n contains v_{n-1} and v_n .

Problem 4.4.2. Show that if G is Eulerian, then it is connected and every vertex has even degree.

Theorem 4.4.3. A graph is Eulerian if and only if it is connected, and every vertex has even degree.

Proof. You showed the easy direction above. Now assume that G is connected and every vertex has even degree. We show that it has a closed Eulerian tour.

Indeed, as a length 1 walk uses no edges, there are walks in G that use no edge more than once. Let

$$W = v_0, e_1, v_1, e_2, \dots, v_{n-1}, e_n, v_n$$

be the longest such walk. We will show that W is a closed Eulerian tour.

Claim 4.4.4. $v_n = v_0$

Proof. If not, the vertex v_n occurs only in an odd number of edges in W . As it has even degree, there is some edge $e_{n+1} = \{v_n, v_{n+1}\}$ that is not used in W . But then W can be extended by e_{n+1} , which contradicts the choice of W . $\diamond$

Claim 4.4.5. All vertices of G are in W .

Proof. If some u is not in W , then as G is connected, there is a path P from u to some $v_i \in W$, none of whose edges are in W . But then the walk

$$u \xrightarrow{P} v_i, e_{i+1}, v_{i+1}, \dots, e_n, v_0, e_1, v_1, \dots, e_i, v_i$$

is a longer walk in G than W , contradicting the choice of W . $\diamond$

Claim 4.4.6. All edge of G are used in W .

Proof. If some edge e is not used, then as W contains all vertices, $e = \{v_i, v_j\}$ for some v_i and v_j in W . We may assume that $i < j$. But then

$$v_i, e_{i+1}, \dots, v_j, e, v_i, e_i, v_{i-1}, \dots, v_0, e_n, v_n, \dots, e_{j+1}, v_j$$

is a longer walk in G than W . $\diamond$

Thus W is a closed Eulerian tour of G . □

Observe that the same proof works if G is allowed to have loops or multiedges.

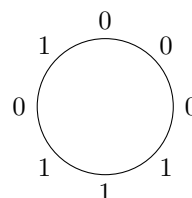
Problem 4.4.7. What is the computational complexity of deciding if a graph on n vertices has an Eulerian tour?

Problems from the Text

Section 4.4: 1,2,5,6,7

4.5 Eulerian Digraphs

In the figure we have arranged several copies of the digits 0 and 1 around a circle so that the eight configurations of three consecutive digits in the clockwise direction are all possible strings in $\{0, 1\}^3$.



Can we do this for all strings in A^n for any alphabet $A = \{a_1, \dots, a_k\}$? This would consist of k^n strings, so would require us to put k^n letters around the circle. Each letter occurs in the same number of strings— k for each time it appears, so each letter will need to be written k^{n-1} times.

We show how to do this using a digraph.

A *digraph* G , short for *directed graph*, consists of a set V of vertices and as set $E \subset V \times V$ of edges. The edges are oriented now, so we write $u \rightarrow v$ to mean that $(u, v) \in E$. For a digraph $G = (V, E)$ the *symmetric graph* $\text{sym}(G)$ underlying G is the graph with vertex set V and edgeset

$$\overline{E} = \{\{u, v\} \mid (u, v) \in E\}.$$

A digraph is *connected* if $\text{sym}(G)$ is connected. A *directed closed Eulerian tour*

$$v_1, v_2, \dots, v_m$$

in G is a closed Eulerian tour in $\text{sym}(G)$ such that $v_i \rightarrow v_{i+1}$ in G for each i , and $v_m \rightarrow v_1$. A digraph G is *Eulerian* if it contains a directed closed Eulerian tour.

The *indegree* of a vertex v in G is

$$\deg^-(v) = \|\{u: u \rightarrow v\}\|$$

and the *outdegree* is

$$\deg^+(v) = \|\{u: v \rightarrow u\}\|.$$

With much the same proof that we used for Euler's Theorem in the previous section we can prove the following digraph version.

Theorem 4.5.1. *A connected digraph G is Eulerian if and only if for every vertex v we have*

$$\deg^-(v) = \deg^+(v).$$

The Debruijn graph $D = D_A^k$ for an alphabet $A = \{a_1, \dots, a_n\}$ is defined as follows. The vertices are the set $V(D) = A^{k-1}$ of strings of length $k-1$ over the alphabet. For every choice of elements $x_1, \dots, x_k \in A$ we have an edge

$$(x_1, \dots, x_{k-1}) \rightarrow (x_2, \dots, x_k);$$

we refer to this edge as $[x_1, \dots, x_k]$.

Problem 4.5.2. Draw the Debruijn graph $D_{\{0,1,2\}}^3$.

Problem 4.5.3. Show that D_A^k has $n^{(k-1)}$ vertices n^k edges, and that every vertex has indegree and outdegree k . Show from any vertex there is a path of length at most k to the vertex $(0, 0, \dots, 0)$, and conclude that D_A^k is connected.

Applying the digraph Euler's theorem we get that D_A^k contains a closed directed Eulerian trail T . We can list the edges of the trail in the order they occur in the trail starting at the edge $[0, 0, \dots, 0]$. Making n^k spots around the circle, write the digits $0, 0, \dots, 0$ in the first k spots. The next edge in T is $[0, 0, \dots, 0, x_1]$ for some $x_1 \in A$. Write this in the $k+1^{th}$ spot, on the circle, so the strings $[0, \dots, 0]$ and $[0, 0, \dots, 0, x_1]$ now appear around the circle. Proceeding around T , each edge e introduces a new letter in its last place, write this in the next spot in the circle, the string e then appears on the circle. By the time we get back to $[0, \dots, 0]$, we have all edges, so all strings in A^k appearing once around the circle.

Problems from the Text

Section 4.5: 1, 2, 3, 4, 5, 6, 8, 9

4.7 Triangle free graphs

Problems in extremal graph theory ask such things as how big or small a graph can be and have a certain property.

Some simple such questions are:

Problem 4.7.1. How many edges can a graph on n vertices have? How few?

Too simple. How about:

Problem 4.7.2. How many edges can a graph on n vertices have and not be connected? How few edges can a graph on n vertices have and be connected?

Still too simple?

Problem 4.7.3. How many edges can a graph on n vertices have, and have no triangles?

The bipartite graph $K_{a,(n-a)}$ has no triangles and has $a(n-a)$ edges.

Problem 4.7.4. Show that $a(n-a)$ is maximized when $a = \lfloor n/2 \rfloor$. Show that $\lfloor n/2 \rfloor (n - \lfloor n/2 \rfloor) = \lfloor n/2 \rfloor \lceil n/2 \rceil = \lfloor n^2/4 \rfloor$.

So a graph can have $\lfloor n^2/4 \rfloor$ edges and have no triangles. Can it have more?

Definition 4.7.5. Let $T(n)$ be the maximum number of edges in a triangle free graph on n vertices.

The above example showed that $T(n) \geq \lfloor n^2/4 \rfloor$.

Theorem 4.7.6. For all $n \geq 1$ we have $T(n) \leq \lfloor n^2/4 \rfloor$.

Proof. As $T(n)$ is an integer, it is enough to show that $T(n) \leq n^2/4$. The base case is $n = 1$ and 2 for which it is clearly true. Assume the theorem holds for all graphs on at least n vertices, and let $G = (V, E)$ be a triangle free graph on $n + 2$ vertices.

Choose an edge $e = \{x, y\}$ from E , and let $G' = (V', E')$ be the subgraph of G induced by $V' = V \setminus \{x, y\}$. Let E_x be the set of edges from x to a vertex of V' , and E_y be the set of edges from y to a vertex of V' .

As $E = E' + E_x + E_y + \{e\}$, and from the fact that G is triangle free we get that $|E_x| + |E_y| \leq n$, we get from the induction hypothesis that

$$|E| \leq \frac{n^2}{4} + n + 1 = \frac{(n+2)^2}{4}.$$

□

Problem 4.7.7. Observing what must happen to get equality in the last line of the proof, prove that if a triangle free graph G on n vertices has $T(n)$ edges, then it is the graph $K_{\lfloor n/2 \rfloor, \lceil n/2 \rceil}$.

5 Trees

5.1 Definition and Characterisations of Trees

Definition 5.1.1. A *tree* is a connected graph containing no cycles.

There are several useful alternate definitions of trees.

Theorem 5.1.2. Let $T = (V, E)$ be a graph with n vertices. Then the following are equivalent.

- i) T is a tree.
- ii) For any two vertices u and v in V there is a unique path between u and v .
- iii) T is connected, but becomes disconnected when we remove any edge.
- iv) T has no cycle, but contains a cycle if we add any edge.
- v) T is connected and $|V| = |E| + 1$.

Draw a couple of trees, and it become pretty clear that the above is true. But proving it takes some care.

First we prove the equivalence of the first four items.

Proof. Assume (i), that T is connected and has no cycles. We show (ii), that there is a unique path between any two vertices. Indeed, as T is connected there must be a path. If there are two paths from u to v then follow one forward and then the other backwards, drawing arrows on the edge as you follow them. This gives you a closed walk from u back to u , in which every path has the same number of in-arrows as out-arrows. If the paths are not the same, you can find an edge $x \rightarrow y$ with an arrow one way, but not the other. Starting at x follow that arrow to y . As y has the same number of in and out arrows, it has a edge with an out-arrow but not an in-arrow. Follow that. In the same way, you can keep following edges with out-arrows but not in arrows, until you end up at a vertex you've already visited. So you have found a cycle, which T does not have. So the paths are the same, and so there is a unique path between u and v .

Assume (ii). We show (iii), that T is connected but removing an edge disconnects it. As there is a path between any two vertices T is connected. If we remove an edge $\{u, v\}$ then we have removed the unique path between u and v , so T becomes disconnected.

Assume (iii). We show (iv), that T has no cycles but does if we add any edge. To see that T has no cycles, assume that it has one containing the edge $e = \{u, v\}$. So there are two paths in T from u to v : one is e , and the other is

the path we get from the cycle by removing e . Now for any pair of vertices x and y in T , if the path between them in T doesn't use e , then it is still in $T - e$, and if it does use e , then there is another walk between them using P instead of e , and this is in $T - e$. This contradicts the fact that $T - e$ is disconnected. To see that adding any edge $e = \{u, v\}$ to T makes a cycle, observe that as T was connected there was already another path from u to v .

Assume (iv), we show (i) by showing that T is connected. Let u and v be in T . Adding the edge $e = \{u, v\}$ we get a cycle in $T + e$, so there was already a path from u to v in T . $\square$

To prove the equivalence of the last item, we start with two Lemma that will be useful in applying induction.

An *end-vertex* or *leaf* of a graph is a vertex of degree one.

Lemma 5.1.3. *Every tree with at least one edge has at least two leaves.*

Proof. If a tree T has at least one edge, then it contains a path, and so contains a maximum path P .

Problem 5.1.4. Show that the endpoints of the path P must be leaves of T . $\square$

Lemma 5.1.5. *Let ℓ be a leaf of a graph T . Then T is a tree if and only if $T - \ell$ is.*

Proof. If T is a tree, then it has no cycles, so $T - \ell$ has no cycles, so $T - \ell$ has no cycles. We are done if we can show that $T - \ell$ is connected. For u and v in $V(T) - \ell$ we have a path P between u and v in T . If this path is not in $T - \ell$ then it contains ℓ . But this is impossible as ℓ has degree 1 and the only degree one vertices in P are the endpoints u and v .

If $T - \ell$ is a tree then it is connected, and so as ℓ has an edge, T is clearly connected. Also $T - \ell$ has no cycles. We must show that T has no cycles. If it had a cycle C , then C would have to contain ℓ . But a cycle can contain no vertex of degree one. $\square$

Finally we finish the proof of Theorem 5.1.2

Proof continued. By induction on $n = |V(T)|$ we prove the equivalence of (i) and (v). When $n = 1$ both (i) and (v) are always true, so the base case is trivial. Assume now that $n > 1$ and the equivalence holds for graphs on at most $n - 1$ vertices.

Assume (i), that T is a tree. Then T has a leaf ℓ and removing it gives a tree T' . By induction $|V(T')| = |E(T')| + 1$. But T has one more vertex and one more edge than T' , so

$$|V(T)| = |V(T')| + 1 = |E(T')| + 2 = |E(T)| + 1,$$

as needed.

Assume (v). Then T is connected and $|V(T)| = |E(T)| + 1$. If every vertex had degree at least 2 then $2|E(T)| = \sum_{v \in V} \deg(v) \geq \sum 2 = 2|V(T)|$ and so $|E(T)| \geq |V(T)|$. But this isn't true so there is some vertex ℓ with degree less than 2. As T is connected and $n > 1$, ℓ has degree 1, so is a leaf. But then where $T' = T - \ell$ we have that

$$|V(T')| = |V(T)| - 1 = |E(T)| + 1 - 1 = |E(T')| + 1$$

and so by induction T' is a tree. But then by Lemma 5.1.5, so is T , as needed. $\square$

Problems from the Text

Section 5.1: 1, 2, 4, 5

5.2 Isomorphism of Trees

The problem of deciding if two graphs are isomorphic is a hard problem. If you are given two graphs G and H on n vertices, and asked to decide if they are isomorphic, you can do it. For each of the $n!$ bijections $\phi : V(G) \rightarrow V(H)$, you can check whether $\phi(g) \sim \phi(g')$ if and only if $g \sim g'$. But this takes a long time. The problem is not NP -complete, but the best algorithm has running time that is superpolynomial in n .

It is easier for trees though. In this section, we show how to decide, quickly—in time $O(n)$, whether two given trees on n vertices are isomorphic.

The algorithm will produce a code $A(T) \in \{0, 1\}^{2^n}$ for any tree on n vertices. This code will have the property that

$$A(T) = A(T') \iff T \cong T'.$$

Clearly we can check whether $A(T) = A(T')$ in time $O(n)$. So it will be enough to see that we can construct $A(T)$ and $A(T')$ in time $O(n)$.

We first construct $A(T)$ for what are called planted trees. So we start with some definitions.

A *rooted tree* (T, r) consists of a tree T and a designated root r . We draw a rooted tree with the root on the top, and the edges going down the the paper, so that the further the vertex is from the root, the lower it is on the paper.

From the name 'root' it feels like we are drawing it upside down, but it is just much easier to draw it this way, as we usually start from the root. And drawing it down jibes with the following definitions. The neighbour of a vertex of a rooted tree that is closer to the root is called its *father*. The neighbours that are further from the root than the vertex are its *children*.

A *planted tree* (T, r, ν) is a rooted (T, r) with a fixed ordering $\nu(v)$ of the children of each (non-leaf) vertex v . When drawing the tree, this is usually represent by drawing the children from left to right according to the order.

We say that two rooted trees (T, r) and (T', r') are isomorphic if there is an isomorphism $\phi : V(T) \rightarrow V(T')$ of T to T' such that $\phi(r) = r'$. Planted trees (T, r, ν) and (T', r', ν') are isomorphic if there is a rooted tree isomorphism $\phi : V(T) \rightarrow V(T')$ such that for children a and b of a vertex v we have that $\phi(a) \leq_{\nu\phi(v)} \phi(b)$ if and only if $a \leq_{\nu(v)} b$.

5.2.1 Determining if two planted trees are isomorphic

For a planted tree (T, r, ν) we define the code $A(T) \in \{a, z\}^{2n}$ recursively, as follows.

- i) Each leaf v gets a label $L_v = [a z]$.
- ii) Let v be a leaf all of whose children $v_1 \leq v_2 \leq \dots \leq v_d$ have labels. Give v the label $L_v = [a L_{v_1} L_{v_2} \dots L_{v_d} z]$.
- iii) When all vertices have labels, let $A(T) = L_r$ be the label of the root.

One important property is the following. (We really wanted to use the symbols $\{-1, 1\}$ instead of $\{a, z\}$, but this makes the concatenation confusing.)

Problem 5.2.1. Let $L_v = x_1 x_2 \dots x_{2d}$ be the labelling of a vertex in the above algorithm. Replace every a with a $' - 1'$ and every z with a $' 1'$. Show that for all m with $1 \leq m \leq 2d$ we have that $\sum_{i=1}^m x_i \geq 0$, and that we have equality if and only if $m = 2d$.

We will use this in the first version of our proof.

Fact 5.2.2. Let (T, r, ν) and (T', r', ν) be planted trees. The codes $A(T)$ and $A(T')$ are the same if and only if (T, r, ν) and (T', r', ν) are isomorphic.

Proof. It is clear that isomorphic planted trees have the same code. We show that the converse is also true by showing that we can recover T from $A(T)$. (This shows that no two trees have the same code.)

Let A be the code of length $2n$ that we get from some planted tree T . We proceed by induction the length of the code. If $n = 1$, the code is $[az]$, and T

is a single vertex. Assume then that n is larger than 1 and any code of length less than $2n$ comes from a unique planted tree.

By definition A begins with a and ends with z , and is of the form $A(T) = [aL_1L_2 \dots L_dz]$ where each L_i is the label of some vertex in a planted tree. By Problem 5.2.1, we can determine exactly what the substrings L_i are: Remove the initial a and the final z . Sum the initial labels (with $a = 1$ and $z = -1$) until getting a 0. At this point we have L_1 . Then continue in the same way, breaking off a L_i every time the sum gets to 0.

By induction, each of the labels L_i is the code for a unique planted tree T_i . So A is the code for the tree T that has a root t , whose children are the roots of the T_i in ascending order.

□

5.2.2 For rooted trees

From a tree T one can make many planted trees, but the planted trees one gets from non-isomorphic tree must be non-isomorphic. So to finish the proof, we need only show that there is a canonical way to get a planted tree (T, r, ν) from a tree T . (Canonical, here, means 'unchanged under isomorphism of T ')

The first step is to define a canonical planted tree from any rooted tree; so we have to canonically order the children of any vertex.

The *lexicographic ordering* of the set of strings over an ordered set A is defined for strings $s = s_1s_2 \dots s_n$ and $t = t_1t_2 \dots t_m$ by setting $s \leq t$ if

- i) s is an initial substring of t , ie, if $t = s_1s_2 \dots s_m$ where $m < n$, or
- ii) the min i for which $s_i \neq t_i$ has $s_i < t_i$.

This is called lexicographic because this is the ordering of words in lexicons (dictionaries).

We can replace step ii) in our labelling algorithm above with the following.

- ii') Let v be a leaf all of whose children have labels. Order the children in lexicographic order: $v_1 \leq v_2 \leq \dots \leq v_d$. Give v the label $L_v = [aL_{v_1}L_{v_2} \dots L_{v_d}z]$

The labelling now defines an injection from the set of rooted trees on n vertices to the set $\{a, z\}^2$.

5.2.3 For trees

Finally we would like to define a canonical root for every tree. We can almost do this by finding what is called the *center* of the tree.

The *eccentricity* $\text{ex } v$ of a vertex v of a graph is the maximum distance any other vertex can be from that vertex. The *center* of a graph is the set of vertices with minimum eccentricity.

Theorem 5.2.3. *Let T be a tree. The center of T consists of a single vertex, or a pair of adjacent vertices.*

Proof. The proof is an induction on the number of vertices of T . It clearly holds if T has one or two vertices. So we may assume that T has at least three vertices, and so it has some non-leaves. Let T' be the non-empty tree we get from T by removing all of its leaves. By induction, T has some center C consisting of a single vertex or a pair of adjacent vertices. We claim that C is also the center of T , and so we are done.

Indeed, for any vertex v of T' we have that the furthest vertex from it must be a leaf, and so for EVERY vertex v of T' we have

$$\text{ex}_T(v) = \text{ex}_{T'}(v) + 1.$$

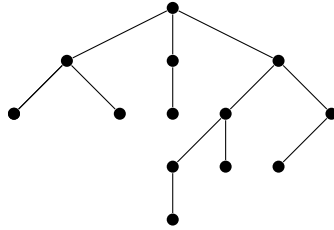
While any vertex of $V(T) \setminus V(T')$ is a leaf ℓ of $V(T)$ so has eccentricity greater than that of its neighbour in T' . $\square$

Now to define the code $A(T)$ for a tree T :

- i) If the center of T is a single vertex r then let $A(T)$ be the code for (T, r) .
- ii) If the center of T consists of two adjacent vertices r and r' let $A(T)$ be the lexicographic minimum of the codes for (T, r) and (T, r') .

Either way, our code we can clearly recover T from its planting, so we can recover it from its code. Thus $A(T) = A(T') \iff T \cong T'$, as needed.

Problem 5.2.4. Find $A(T)$ for the following planted tree T . Then, treating the tree as unrooted, find its center, and find $A(T)$ for it as a tree.



Problems from the Text

Section 5.2: 1, 2, 3(a), 6

5.4 The Minimum Spanning Tree Problem

Definition 5.4.1. A *spanning tree* of a graph G is a subgraph with $|V(G)|$ vertices that is isomorphic to a tree.

It is not hard to show that a graph G is connected if and only if it has a spanning tree. Indeed in a connected graph G with n vertices, we can find a spanning tree T with the following algorithm.

- i) Choose a first vertex v_0 .
- ii) For i from 1 to $n - 1$, choose a vertex v_i from $V(G) \setminus \{v_0, \dots, v_{i-1}\}$ with a neighbour $w_i \in \{v_0, \dots, v_{i-1}\}$ and let $\{v_i, w_i\}$ be in T .

As G is connected there will always be some v_i in $V(G) \setminus \{v_0, \dots, v_{i-1}\}$ with a neighbour in $\{v_0, \dots, v_{i-1}\}$. The algorithm chooses $n - 1$ edges for T , and one can check by induction that $T|_{\{v_0, \dots, v_i\}}$ is connected for all i , so T is connected subgraph of G on n vertices having $n - 1$ edges. Thus it is a spanning tree of G .

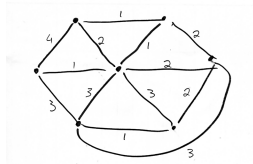
Problem 5.4.2. In the above algorithm, show that $T|_{\{v_0, \dots, v_i\}}$ is connected for all i .

That was easy, but sometimes certain spanning trees are better than others.

Definition 5.4.3. A *weighted graph* consists of a graph $G = (V, E)$ along with a *weight function* $w : V \rightarrow \mathbb{R}$. For a subgraph G' of G the weight $w(G')$ of G' is the sum

$$w(G') = \sum_{e \in E(G')} w(e).$$

Problem 5.4.4. Find a minimum weight spanning tree of the weighted graph shown below.



This problem can be a bit harder than simply finding any spanning tree. But it is not so much harder.

The following is Kruskal's algorithm for finding a minimum weight spanning tree T in a connected weighted graph G .

- i) Order the edges of G so that $w(e_0) \leq w(e_1) \leq \dots \leq w(e_{m-1})$.

ii) Let $e_1 \in T$.

iii) For $i = 1$ to $m - 1$ put e_i into T unless it makes a cycle.

Problem 5.4.5. Use Kruskal's algorithm to find a spanning tree of the graph in the previous problem.

Problem 5.4.6. Show that the graph T returned by the algorithm is a spanning tree.

Proposition 5.4.7. *The spanning tree T given by Kruskal's algorithm is a minimum weight spanning tree.*

Proof. Towards contradiction, assume that there is a spanning tree S of G with $w(S) < w(T)$. Relabel the edges of T as $\{t_1, \dots, t_n\}$ in non-decreasing order of weight; that is, so

$$w(t_1) \leq w(t_2) \leq \dots \leq w(t_n).$$

Similarly relabel the edges of S as $\{s_1, \dots, s_n\}$ in non-decreasing order of weight. As $w(S) < w(T)$ there must be some i so that $w(s_i) < w(t_i)$. Fix i as the minimum such index.

Let T' be the subgraph of T consisting of the edges $\{t_1, \dots, t_{i-1}\}$ and S' be the subgraph of S consisting of the edges $\{s_1, \dots, s_i\}$. As both are subgraphs of trees, neither contains a cycle.

Claim 5.4.8. *There is an edge $s \in S'$ whose vertices are in different components of T' .*

Proof. Let $T_1, \dots, T_r$ be the components of T' . As S' is acyclic, the number of possible edges of S' between vertices of T_j is at most $|V(T_j)| - 1$, and as T_j is a tree, the number of edges are exactly $|E(T_j)| = |V(T_j)| - 1$. So S' has at most as many edges among vertices of T_j as T does. But S' has more edges than T' overall, so some edge of S' must be between components of T' . $\diamond$

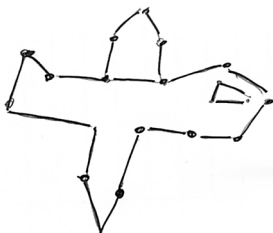
As this $s \in \{s_1, \dots, s_i\}$ we have $w(s) \leq w(s_i) < w(t_i)$. But then our algorithm should have chosen s instead of t_i to add to T . This is a contradiction. $\square$

An algorithm, such as Kruskal's algorithm, that take the best edge it can see at every step, is called a *greedy algorithm*. Very often greedy algorithms don't work. This is one of the nice cases that they do.

Problems from the Text

Section 5.4: 1,2,3,4,5,8

6 Drawing graphs in the plane



That's drawing planes in the graph, smart ass!

6.1 Drawing in the plane and on other surfaces

A graph G is *planar* if it can be drawn in the plane with no edges crossing. A drawing of the graph in the plane with no edges crossing is called a *topological planar graph* or a *plane graph*.

Problem 6.1.1. Show that K_4 is planar but that K_5 is not.

We can talk of drawing graphs on other surfaces, such the spheres, the torus, or the double torus.

Problem 6.1.2. Show that K_5 can be drawn on the torus (doughnut).

Problems from the Text

Section 6.1: 1, 2

6.2 Cycles in Planar graphs

We showed that K_5 is not planar by ad-hoc arguments, but such arguments difficult for larger graphs. That's okay though. That K_5 is not planar tells us that a lot of other graphs are not planar.

Problem 6.2.1. Show that no graph containing K_5 as a subgraph, can be planar. Show that the Petersen graph is not planar.

Definition 6.2.2. A *subdivision* of a graph G is a graph we get from G by dividing edges. That is, G' is a subdivision of G if we can construct it by repeated application of the following construction, called an *edge subdivision*.

- remove an edge $e = ab$,

- add a new vertex v_e ,
- add the edges av_e and bv_e .

It should be pretty clear that because K_5 is not planar, no subdivision of K_5 is planar. So no graph containing a subdivision of K_5 as a subgraph is planar. Can you think of any other graphs that are not planar? Please don't look ahead. It will ruin the surprise.

Okay. Here's the surprise. $K_{3,3}$ is also not planar, and has no subdivision of K_5 . But that is essentially it. Kuratowski showed the following.

Theorem 6.2.3 (Kuratowski's Theorem). *A graph is planar if and only if it contains no subdivision of K_5 or $K_{3,3}$ as a subgraph.*

We won't prove this, as it is a little involved. But we should show the following.

Problem 6.2.4. Show that $K_{3,3}$ is not planar.

One can do this with ad-hoc arguments. But we can also do it more methodically. We do so in the next section.

Problems from the Text

Section 6.2: 2

6.3 Euler's Formula

A planar drawing of a graph G defines *faces* in the plane: the little bits of plane you get by cutting along all the edges of the drawing. Where $|V(G)| = n$ and $|E(G)| = m$ and f is the number of faces. Euler showed the following fundamental identity.

Theorem 6.3.1 (Euler's Formula). *For any embedding of a connected graph G in the plane,*

$$n - m + f = 2.$$

Proof. The proof is by induction on m . As G is connected, $m \geq n - 1$. If $m = n - 1$ then G is a tree, so there is only one face, and the identity holds. This is our base case.

Now assume that $m \geq n$. Then there is an edge e whose removal does not disconnect the graph. Remove it. Its removal joins two faces, so reduces f by one, but also reduces m by one. So the identity holds from the induction hypothesis. $\square$

Corollary 6.3.2. *If G is connected and planar and $n \geq 3$, then*

$$m \leq 3n - 6.$$

Proof. Count the number N of pairs (e, r) where e is an edge, and r is a face with e on the boundary. As every edge is in at most 2 faces, we have that $N < 2m$ and as every face has at least 3 edges, $3f < N$ so

$$f \leq N/3 \leq \frac{2}{3}m.$$

Plugging this into $n - m + f = 2$ gives that

$$n - m/3 = n - m + \frac{2}{3}m \geq 2$$

so $m \leq 3n - 6$ as needed. $\square$

Problem 6.3.3. Use this corollary to show that K_5 is not planar.

Problem 6.3.4. Using the fact that a bipartite graph has no triangles, so a face of drawing must have at least four boundary edges, show that for a bipartite planar graph

$$m \leq 2n - 4$$

. Use this to show that $K_{3,3}$ is not planar.

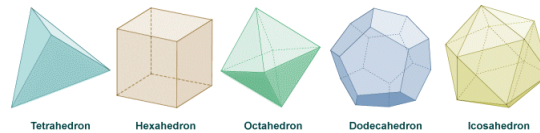
Problem 6.3.5. Let G be a planar drawing of graph with 16 edges in which every vertex has degree 4. How many faces does the drawing make?

Problem 6.3.6. Show that any planar graph has a vertex of degree at most 5.

Platonic Solids (a bit of fun)

Recall that a *platonic solid* is a 3-dimensional convex solid bounded by f copies of the same flat face, such that every vertex (point of intersection of more than two faces) intersects the same number of faces.

There are 5 platonic solids:



It isn't the original proof, but Euler's formula gives a nice proof that these are the only platonic solids.

The set of vertices of such a solid and the set of edges (intersections of 2 faces) make up a regular graph, and it is not too hard that this graph can be drawn in the plane so that every face has the same number of bounding edges.

(Recall a graph is regular if every vertex has the same degree.)

Theorem 6.3.7. *The only connected regular graphs that can be drawn in the plane with each face having the same number of edges, are the 5 we get from the platonic solids above, and cycles.*

Proof. Let G be a planar graph that is regular with degree d , and assume there is a planar drawing such that every face has b edges. Let f be the number of faces.

We have seen that the sum of the degrees of a graph is $2m$, so

$$2m = dn.$$

Summing the number of edges per face over all the faces, we get fb and clearly every edges was counted twice, so

$$fb = 2m = 2n.$$

By Euler's formula we then get that

$$2 = n - m + f = \frac{2m}{d} - m + \frac{2m}{b}$$

yielding

$$\frac{1}{m} = \frac{1}{d} - \frac{1}{2} + \frac{1}{b}$$

and so

$$\frac{1}{d} + \frac{1}{b} = \frac{1}{2} + \frac{1}{m}.$$

If $b, d \geq 4$ then $\frac{1}{d} + \frac{1}{b} \leq \frac{1}{2}$ so there is no solution to above equation. If one of b or d is 3 and the other is at least 6, the again there is no solution. (b can never be less than 3 as there is no face with 2 edges; if d is less than 3 then G can only be a cycle.) If $b = 3$ then the d can be 3 (tetrahedron), 4 (octahedron) or 5 (icosohedron). If $d = 3$ then b can be 3 (tetrahedron), 4 (cube), or 5 (dodecahedron). $\square$

Problems from the Text

Section 6.3: 1,2,3,7(for fun)

9 Finite Projective Planes

Before we actually get into this section we add some background material that we will use.

Modular Arithmetic

9.0.1 The Euclidean Algorithm

Recall that for any two integers n and $m > 0$ there are unique integers q and r , with $0 \leq r < m$ such that

$$n = q(m) + r.$$

Usually q is called the *quotient* of n by m , and r is the *remainder* when dividing n by m . We call r the *image of n modulo (or mod) m* .

An integer d *divides* a , or is a divisor of a if the image of a modulo d is 0.

Problem 9.0.1. Show that d divides a if and only if there is some q such that $a = qd$.

Problem 9.0.2. Show that if d divides m and n then it divides r where $n = q(m) + r$.

For integers a and b , the *greatest common division* $\gcd(a, b)$ of a and b is the largest integer d that divides a and b .

The extended Euclidean algorithm tells us that for any integers a and b , if there are integers p and q such that

$$p(a) + q(b) = \gcd(a, b).$$

In particular if m is prime then for any integer i with $0 < i < m$ there are integers p and q such that

$$p(a) + q(b) = 1.$$

9.0.2 Definition and properties of Modular Integers

Let $\mathbb{Z}_m = \{0, 1, \dots, m-1\}$ be the set of possible images of integers mod m . We can make $\mathbb{Z}_m$ into what is called a ring by defining operations. For $x, y \in \mathbb{Z}_m$ let

$$x \oplus y = x + y \mod m$$

and let

$$x \otimes y = x \times y \mod m.$$

So for example in $\mathbb{Z}_7$ we have that $5 \oplus 2 = 0$, and $5 \oplus 4 = 2$.

Problem 9.0.3. Fill in the addition and multiplication tables for $\oplus$ and $\otimes$ in $\mathbb{Z}_4$.

$\oplus$	0	1	2	3
0	0	1		3
1				0
2				
3				

$\otimes$	0	1	2	3
0	0	0	0	
1				
2		2		
3				1

The operations $\oplus$ and $\otimes$ have many of the same properties that $+$ and $\times$ have for $\mathbb{Z}$.

Problem 9.0.4. Show the following for a, b and c in $\mathbb{Z}_m$.

- $a \oplus b = b \oplus a$
- $a \otimes b = b \otimes a$
- $0 \oplus a = a$
- $0 \otimes a = 0$
- $1 \otimes a = a$
- $a \otimes (b \oplus c) = (a \otimes b) \oplus (a \otimes c)$

Not everything is the same though.

Example 9.0.5. We do not have cancellation in $\mathbb{Z}_6$. Indeed we have that

$$2 \otimes 1 = 2 = 2 \otimes 4,$$

but $1 \neq 4$. We cannot cancel the 2. The problem is that $2 \otimes 3 = 0$.

A *zero-divisor* in $\mathbb{Z}_m$ is a number $a \neq 0$ such that $a \otimes b = 0$ for some $b \in \mathbb{Z}_m$ with $b \neq 0$.

There are no zero-divisors in $\mathbb{Z}$, but there may be in $\mathbb{Z}_m$.

Problem 9.0.6. Show that if m is not prime then $\mathbb{Z}_m$ has zero-divisors.

A *multiplicative inverse* of an element $a \in \mathbb{Z}_m$ is a number b such that $b \oplus a = 1$. Because of the following, we may denote the multiplicative inverse of a by a^{-1} .

Problem 9.0.7. Show that if $a \in \mathbb{Z}_m$ has a multiplicative inverse, then it has only one of them.

Problem 9.0.8. Show that if a has a multiplicative inverse then it is not a zero-divisor.

Theorem 9.0.9. *The ring $\mathbb{Z}_m$ contains zero-divisors if and only if m is prime.*

Proof. You have already shown the only if part. For the if part, observe that if m is prime, then for any non-zero integer $a \in \mathbb{Z}_m$ we have by the Extended Euclidean Algorithm that

$$1 = x(m) + y(a),$$

so $ya = xm + 1$. Thus $y \otimes a = 1$ which means that $y = a^{-1}$ is the multiplicative inverse of a . By the previous problem, a is therefore not a zero-divisor. This is true for all $a \neq 0$, and so $\mathbb{Z}_m$ has no zero-divisors. $\square$

As there are no problems in the text for this section, here are some.

Problem 9.0.10. Find the image of $15(17) - 19$ in $\mathbb{Z}_7$.

Problem 9.0.11. Find the last digit of 7^{100} .

Problem 9.0.12. Find the multiplicative inverse of 5 in $\mathbb{Z}_9$.

Problem 9.0.13. Show that for any n , every element $a \in \mathbb{Z}_n$ has an additive inverse: an element $-a$ such that $a \oplus (-a) = 0$. Show that if $a \oplus b = a \oplus c$ then $b = c$. (Usually we write $a \ominus b$ for $a \oplus (-b)$).

Problem 9.0.14. Show that for any n and any $a \in \mathbb{Z}_n$ the set

$$\mathbb{Z}_n \oplus a = \{x \oplus a \mid x \in \mathbb{Z}_n\}$$

is $\mathbb{Z}_n$.

Problem 9.0.15. Show that for $a \in \mathbb{Z}_n$ the set

$$\mathbb{Z}_n \otimes a = \{x \otimes a \mid x \in \mathbb{Z}_m\}$$

is $\mathbb{Z}_n$ if and only if a has a multiplicative inverse in $\mathbb{Z}_n$.

Problem 9.0.16. Show that 3 divides n if and only if 3 divides the sum of the digits of n .

Problem 9.0.17. Show that for all $n \geq 1$, $4^n = 3n + 1$ in $\mathbb{Z}_9$. (Hint: use induction)

9.1 Definitions and basic properties

Recall that in a plane any two points determine a unique line, and any two lines intersect in a point. Oh, unless they are parallel! Perhaps you have seen that this 'unless they are parallel' exception can be removed by adding 'points at infinity' where parallel lines meet. This is what they do to make the real projective plane.

This is too big for us. We want a discrete version of a projective plane.

Definition 9.1.1. Let X be a finite set of elements (called *points*) and let $\mathcal{L}$ be a family of subsets (called *lines*) of X . The pair $(X, \mathcal{L})$ is a *projective plane* if the following *axioms* are true:

- (P0) There is a 4 element set $F \subset X$ such that $|L \cap F| \leq 2$ for each $L \in \mathcal{L}$.
- (P1) Any two distinct sets L and L' in $\mathcal{L}$ have $|L \cap L'| = 1$.
- (P2) For any two distinct elements $x_1, x_2 \in X$ there is exactly one set in $\mathcal{L}$ containing x_1 and x_2 .

Axioms (P1) and (P2) are analogues of the properties we mentioned above for projective planes. Axiom (P0) is just there to remove some uninteresting *degenerate* structures that satisfy the other two conditions: structures such as empty structures, a single point with many (or no) lines, a line with many (or no) points, a structure with 3 points.

It turns out, that these axioms are quite restrictive. Finite projective planes are a bit hard to find. Can you find one? Start with the four points you have by axiom (P0), and see what it forces. Drawing lines in a plane is useful, but also dangerous.

Lets look at some of the restrictions these axioms imply.

Lemma 9.1.2. *For any two lines of a projective plane $P = (X, \mathcal{L})$, there is some point not in either line.*

Proof. Let L and L' be lines of a projective plane P . Consider the four points $\{a, b, c, d\}$ in X we have by axiom (P0). If one of them is not on L or L' we are done. So each of them is either on L or L' , and by axiom (P0) there are two of them on each of L and L' . We may assume that $\{a, b\} \subset L$ and $\{c, d\} \subset L'$. But then there is some line L_{ac} through a and c and a line L_{bd} through b and d , and again by axiom (P0) they do not intersect in any of a, b, c or d . So they intersect in some point x . If $x \in L$, then there are two lines containing it and a and if $x \in L'$ then there are two lines containing it and c . Neither is allowed. $\square$

Proposition 9.1.3. *In a finite projective plane, all lines contain the same number of points.*

Proof. Let L and L' be two lines of a projective plane $(X, \mathcal{L})$. By the Lemma, there is some point x on neither of them. Now, for any point a in L , a is in a line L_a with x , and this line L_a must intersect L' in exactly one point a' . The map $a \mapsto a'$ is a bijection from L to L' . Indeed, it is clearly an injection, and it is invertible by defining a map from L' to L in the same way. $\square$

With this done, the following is a good definition.

Definition 9.1.4. The order of a projective plane $(X, \mathcal{L})$ is the value

$$n = |\mathcal{L}| - 1$$

for any line $L \in \mathcal{L}$.

For a projective plane $P = (X, \mathcal{L})$ of order n do the following.

Problem 9.1.5. Show that for any point x there is a line L not containing x .

Problem 9.1.6. Show each point of P is in $n + 1$ lines.

Problem 9.1.7. Show $|X| = n^2 + n + 1$.

Problem 9.1.8. Show $|\mathcal{L}| = n^2 + n + 1$.

Definition 9.1.9. Let $(X, \mathcal{L})$ be a projective plane. For a point $x \in X$ let λ_x be the family of lines containing x . The *dual* $(\mathcal{L}, \Lambda)$ of $(X, \mathcal{L})$ is made by viewing the elements of $\mathcal{L}$ as points, and defining the following set of lines:

$$\Lambda = \{\lambda_x \mid x \in X\}.$$

Proposition 9.1.10. *The dual $P' = (\mathcal{L}, \Lambda)$ of a finite projective plane $P = (X, \mathcal{L})$, is a finite projective plane.*

Proof. To show that P' satisfies axiom (P1) let λ_a and λ_b be lines of P' . As a and b occur together in a unique line L_{ab} of P , $\lambda_a \cap \lambda_b = \{L_{ab}\}$.

To show that P' satisfies axiom (P2) let L and L' be distinct points of P' . As L and L' are lines of P they intersect in exactly one point a of P . So they occur together in exactly one line λ_a of P' .

For axiom (P0) we must find four points of P' (so lines of P) such that no three of them are in a line λ_x of P' (so that no three of them intersect in a point x of P). Take the four points a, b, c, d of P assured by axiom P. They define $\binom{4}{2} = 6$ lines each of which contains exactly two of a, b, c, d . Remove the one containing a, b and the one containing c, d and we are left with four lines L_{ac}, L_{ad}, L_{bc} and L_{bd} where the index of each tells us which two points in $\{a, b, c, d\}$ it contains. These are our points: no three of them intersect in a point in $\{a, b, c, d\}$, and no three of them intersect in some other point x , as two of them already intersect in one of the elements of $\{a, b, c, d\}$.

□

So the dual of a dual of a projective plane is also a projective plane.

Proposition 9.1.11. *The dual of the dual of a projective plane P , is P .*

We won't prove this. The proof of these comes immediately from an alternate construction of the dual which we only sketch. It is given in more detail in the text.

For a projective plane $P = (X, \mathcal{L})$ let G_P be the bipartite graph we get on the vertex set $V(G) = X \cup \mathcal{L}$ by putting an edge between $x \in X$ and $L \in \mathcal{L}$ if $x \in L$. Then the dual P' of P is the projective plane whose graph $G_{P'}$ we get from G_P by switching the roles of X and $\mathcal{L}$.

Taking the dual of the dual switches the roles of X and $\mathcal{L}$, and then switches them back.

Problems from the Text

1, 3(ab), 4:

Problem 9.1.12. If a projective plane has each point in 6 lines, how many points does it have?

Problem 9.1.13. If a projective plane has 31 points, how many points lie in each line?

9.2 Existence of finite projective planes

We have seen a projective plane of order 2. Projective planes of order 3, 4 and 5 exist, but there are none of order 6. There exists some of order 7, 8, 9 and 11. There are none of order 12, and we don't know if there is one of order 12. It's a hard problem to decide.

Now we construct some finite projective planes. They are related to latin squares, so we talk about these first.

9.3 Latin Squares

This section is very brief in the text. We expand it some.

9.3.1 Definition and examples

A precursor of Sudoku things, a *latin square of order n* is an $n \times n$ matrix with entries from $\mathbb{Z}_n$ such that each number in $\mathbb{Z}_n$ occurs exactly once in each row and column.

Some easy examples are

0	1
0	1

0	1	2
2	0	1
1	2	0

0	1	2	3
3	0	1	2
2	3	0	1
1	2	3	0

Maybe you saw the last as the addition table for $\oplus$ in $\mathbb{Z}_4$. Indeed, the addition table for $\mathbb{Z}_n$ gives a latin square for all n .

Note

Usually we number the rows and columns of an $n \times n$ matrix by the numbers $i = 1, \dots, n$, but as we are playing with $\mathbb{Z}_n$ a lot, it will be useful to number them with $i = 0, \dots, n-1$. So the upper left entry of $A = [a_{ij}]$ is a_{00} .

Theorem 9.3.1. *The $n \times n$ matrix $A = [a_{ij}]$ where*

$$a_{ij} = i \oplus j$$

is a latin square.

Proof. As there are n symbols, and n entries in each row and column, it is enough to show that no symbol $a \in \mathbb{Z}_n$ occurs twice in the same row or column. Assume for some r that $a_{ri} = a = a_{rj}$. Then

$$r \oplus i = a_{ri} = a = a_{rj} = r \oplus j,$$

which gives that $i = j$. So no entry a occurs twice in a row. A similar proof shows no entry occurs twice in a column. $\square$

So latin squares are pretty easy to find. In fact, as we can permute the symbols $\{0, \dots, n-1\}$ in a latin square, and get another, once we have one we have $n!$ isomorphic ones.

There are plenty of other latin squares too:

0	1	2	3	4
2	3	4	0	1
4	0	1	2	3
1	2	3	4	0
3	4	0	1	2

Problem 9.3.2. For prime p and a non-zero number $a \in \mathbb{Z}_p$ show that the $p \times p$ matrix $A^{(a)} = [a_{ij}]$ where

$$a_{ij} = (i \otimes a) \oplus j$$

is a Latin square. Hint: use problems 9.0.14 and 9.0.15

By permuting the symbols, we can always assume that a latin square is in standard form: the first row is $(0, 1, \dots, n-1)$.

Definition 9.3.3. Two latin squares $A = [a_{ij}]$ and $B = [b_{ij}]$ of order n are *orthogonal* if

$$\{(a_{ij}, b_{ij}) \mid i, j \in [n]\} = \mathbb{Z}_n \times \mathbb{Z}_n.$$

.

Example 9.3.4. The latin squares

0	1	2
1	2	0
2	0	1

and

0	1	2
2	0	1
1	2	0

are orthogonal because when we superimpose them we get each ordered pair

0	1	2
0	1	2
1	2	0
2	0	1
2	0	1
1	2	0
2	0	1

Can we find three latin squares of order 3 any two of which are orthogonal?

Definition 9.3.5. A set $A^{(1)}, \dots, A^{(k)}$ of latin squares are *mutually orthogonal latin squares* (MOLS) if for any $i \neq j \in [k]$ the latin squares $A^{(i)}$ and $A^{(j)}$ are orthogonal.

Theorem 9.3.6. If $A^{(1)}, \dots, A^{(k)}$ is a set of MOLS of order n , then $k < n$.

Proof. Suppose we had MOLS $A^{(k)} = [a_{ij}^{(k)}]$ of order n for $k = 1, \dots, n$. Then we may assume that each is in standard form. So $a_{00}^{(k)} = 0$ for all k . But then $a_{10}^{(k)} \neq 0$ for any k so the set $\{a_{10}^{(1)}, \dots, a_{10}^{(n)}\}$ must contain a repeated element a . Without loss of generality, assume that $a_{10}^{(1)} = a_{10}^{(2)} = 1$. Then $A^{(1)}$ and $A^{(2)}$ are not orthogonal as $a_{01}^{(1)} = a_{01}^{(2)} = 1$ also. $\square$

Now this bound is sharp if p is prime.

Look at the following four latin squares of order 5:

0	1	2	3	4
1	2	3	4	0
2	3	4	0	1
3	4	0	1	2
4	0	1	2	3

0	1	2	3	4
2	3	4	0	1
4	0	1	2	3
1	2	3	4	0
3	4	0	1	2

0	1	2	3	4
3	4	0	1	2
1	2	3	4	0
4	0	1	2	3
2	3	4	0	1

0	1	2	3	4
4	0	1	2	3
3	4	0	1	2
2	3	4	0	1
1	2	3	4	0

Do they look orthogonal?

Theorem 9.3.7. For any prime p , the order p latin squares $A^{(1)}, \dots, A^{(p-1)}$ defined in Problem 9.3.2 are MOLS.

Proof. Fix $a \neq b \in [1, \dots, p-1]$. To show that $A^{(a)}$ and $A^{(b)}$ are orthogonal, it is enough to find for every $(\alpha, \beta) \in \mathbb{Z}_p^2$ a pair (i, j) such that

$$\alpha = a_{ij}^{(a)} = (i \otimes a) \oplus j$$

and

$$\beta = a_{ij}^{(b)} = (i \otimes b) \oplus j.$$

This is two equations in two unknowns.

Problem 9.3.8. Finish the proof by solving these equations for i and j . Check that your solution is right with the $p = 5$ examples above.

□

Problem 9.3.9. Construct two MOLS of order 7

Problem 9.3.10. A latin square $A = [a_{ij}]$ is idempotent if $a_{ii} = i$ for all i . Construct an idempotent latin square of order 5.

9.4 Projective Planes from MOLS

Theorem 9.4.1. *For $n \geq 2$ there is a projective plane of order n if and only if there exists a collection of $n - 1$ MOLS of order n .*

Sketch of proof one way. Let $A^{(1)}, \dots, A^{(n-1)}$ be a family of $n - 1$ MOLS of order n . We construct a projective plane $P = (X, \mathcal{L})$ of order n .

For the points of P let X consist of

- The positions of a latin square: pairs $(i, j) \in \mathbb{Z}_n^2$.
- Points at infinity: c, r and s_i for $i = 1, \dots, n - 1$.

The lines are

- $B = \{c, r, s_1, \dots, s_{n-1}\}$.
- $C_j = \{c, (1, j), (2, j), \dots, (n, j)\}$ for each $j \in \mathbb{Z}_n$.
- $R_i = \{r, (i, 1), (i, 2), \dots, (i, n)\}$ for each $i \in \mathbb{Z}_n$.
- $S_{km} = \{s_k\} \cup \{(i, j) \mid a_{ij}^{(k)} = m\}$ for each $k = 1, \dots, n - 1$ and each $m \in \mathbb{Z}_n$.

The last is the only one that is a bit hard to understand. For each of the latin squares $A^{(k)}$ and each symbol m , we make a line, with s_k of all the positions in which $A^{(k)}$ has an m . □

Problem 9.4.2. Starting with the one latin square of order 2, make a projective plane of order 2.

Problem 9.4.3. Starting with the fano plane, pick an arbitrary edge to be the infinite edge B . Choose points of B to be c, r and s_1 . Show how this defined a latin square.

Problem 9.4.4. Make a projective plan of order 3.

Problems from the Text

Section 6.3: 2,6